

Lesson 2: JavaScript Loops and Arrays

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Explain what a loop is and write a `for` loop and a `while` loop
- Create an array and access items by index
- Use common array methods (`push`, `pop`, `includes`, `slice`)
- Loop through an array to process its items
- Describe what scope means and why it matters

Explore

Why Loops?

Imagine printing "Hello World!" 100 times.

```
console.log("Hello World!");  
console.log("Hello World!");  
// ... 98 more lines
```

That is slow and hard to maintain. A **loop** lets us do this in 3 lines.

Computers never get tired. Loops let us take advantage of that.

The **for** Loop

The most common loop in JavaScript:

```
for (let i = 0; i < 5; i++) {  
  console.log("Hello World!");  
}
```

Three parts in the parentheses:

Part	Code	Meaning
Initialize	<code>let i = 0</code>	start here
Condition	<code>i < 5</code>	keep going while true
Update	<code>i++</code>	move to the next step

The `for` Loop — Step by Step

```
for (let i = 0; i < 3; i++) {  
    console.log("Count:", i);  
}  
// Count: 0  
// Count: 1  
// Count: 2
```

Zoom chat: Why does the output stop at 2, not 3?

The `while` Loop

Use `while` when you don't know the exact number of repetitions up front.

```
let count = 0;

while (count < 3) {
  console.log("Count:", count);
  count++;
}
```

The loop keeps running **as long as** the condition is `true`.

Forgetting `count++` causes an **infinite loop** — the tab freezes!

Check for Understanding

What will this code print?

```
for (let i = 1; i <= 4; i++) {  
  console.log(i);  
}
```

- A) 0 1 2 3
- B) 1 2 3 4
- C) 1 2 3 4 5
- D) Nothing — there is an error

Arrays

An array is an **ordered list** of items.

```
let fruits = ["apple", "banana", "cherry"];
```

Access items by their **index** — counting starts at **0**:

```
console.log(fruits[0]); // "apple"  
console.log(fruits[1]); // "banana"  
console.log(fruits[2]); // "cherry"
```

Zoom chat: What index would you use to get "cherry"?

Array Properties and Methods

```
let colors = ["red", "blue", "green"];

colors.length;           // 3 – how many items
colors[1] = "yellow";    // change an item
colors.push("purple");   // add to the end
colors.pop();            // remove from the end
colors.includes("red");  // true – is "red" in there?
```

length is a property (no parentheses). Methods need ().

Copying an Array with `slice()`

Assigning an array to a new variable does **not** make a copy.

```
let original = [1, 2, 3];  
let copy = original;           // same array, not a copy!  
copy.push(4);  
console.log(original);         // [1, 2, 3, 4] – changed!
```

Use `slice()` to make a true copy:

```
let safeCopy = original.slice();
```

Loops + Arrays

Loops and arrays work together naturally.

```
let fruits = ["apple", "banana", "cherry"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
// apple
// banana
// cherry
```

Using `fruits.length` in the condition means the loop always fits the array — even if the array grows.

Check for Understanding

What does this code print?

```
let nums = [10, 20, 30];  
let total = 0;  
  
for (let i = 0; i < nums.length; i++) {  
    total += nums[i];  
}  
console.log(total);
```

- A) [10, 20, 30]
- B) 0
- C) 60
- D) 30

Scope

Scope controls where a variable can be used.

```
let globalVar = "available everywhere";

function myFunction() {
  let localVar = "only inside here";
  console.log(globalVar);    // works
  console.log(localVar);    // works
}

console.log(globalVar);      // works
console.log(localVar);      // ERROR – not in scope
```

Block Scope

Variables declared with `let` inside a block `{ }` stay inside that block.

```
if (true) {  
  let blockVar = "I live in this block";  
  console.log(blockVar);    // works  
}  
  
console.log(blockVar);      // ERROR – blockVar is gone
```

This is why `let` is safer than `var` — `var` leaks outside blocks.

Check for Understanding

Which line will cause an error?

```
function greet() {  
    let message = "Hello!";  
    console.log(message);    // Line A  
}  
greet();  
console.log(message);    // Line B
```

- A) Line A — `message` doesn't exist yet
- B) Line B — `message` is out of scope
- C) Both lines — `let` cannot be used inside functions
- D) Neither — both lines work fine

Apply

Core Activity: Let's Code Together

We will build functions similar to the assignment — **you tell me what to type!**

Open your browser console:

- **Chrome / Edge:** Right-click anywhere → Inspect → Console
- **Firefox:** Right-click anywhere → Inspect → Console

Step 1 — Repeat with a Loop

Challenge: Write a function that prints "Hello World!" a given number of times.

```
function repeat(times) {  
  for (let i = 0; i < times; i++) {  
    console.log("Hello World!");  
  }  
}  
  
repeat(3);
```

Ask the group: Why is the `console.log` inside the function here instead of outside?

Step 2 — Sum an Array

Challenge: Write a function that adds up all numbers in an array.

```
function calculateTotal(numbers) {  
  let sum = 0;  
  for (let i = 0; i < numbers.length; i++) {  
    sum += numbers[i];  
  }  
  return sum;  
}  
  
console.log("Q10:", calculateTotal([3, 4, 2, 8]));  
// Q10: 17
```

Ask: What does `sum += numbers[i]` do on each step of the loop?

Step 3 — Filter an Array

Challenge: Write a function that returns only the even numbers from an array.

```
function findEvens(numbers) {  
  let evens = [];  
  for (let i = 0; i < numbers.length; i++) {  
    if (numbers[i] % 2 === 0) {  
      evens.push(numbers[i]);  
    }  
  }  
  return evens;  
}  
  
console.log(findEvens([10, 3, 7, 6, 2]));  
// [10, 6, 2]
```

Zoom chat: What does `%` (modulo) do?

Step 4 — Try It Yourself

Pick one to try on your own (5 minutes):

A) Write a function called `pyramidCounting` that takes a number and returns the sum of all numbers from 0 up to that number. `pyramidCounting(4)` should return `10`.

B) Write a function called `displaySkills` that takes an array of strings and prints each one. Test it with `["JavaScript", "HTML", "CSS"]`.

C) Write a function called `makeSquares` that takes an array of numbers and returns a new array with each number squared. `makeSquares([2, 5, 8])` should return `[4, 25, 64]`.

Extension Activity

Only if students are ready and time allows.

Extension — FizzBuzz

A classic interview challenge! Loop from 1 to 15 and build an array:

```
function fizzBuzz() {  
  let result = [];  
  for (let i = 1; i <= 15; i++) {  
    if (i % 3 === 0 && i % 5 === 0) {  
      result.push("fizzbuzz");  
    } else if (i % 3 === 0) {  
      result.push("fizz");  
    } else if (i % 5 === 0) {  
      result.push("buzz");  
    } else {  
      result.push(i);  
    }  
  }  
  return result;  
}
```

Ask: Why must we check "divisible by both" first?

Assignment Preview

15 questions this week, grouped by topic:

Questions	Topic
Q1–Q5	Loop-based functions (repeat, sum, vowels, odds)
Q6–Q9	Array operations (check, access, insert, compare)
Q10–Q12	Arrays + loops (sum, filter, square)
Q13	Display skills from an array
Q14	FizzBuzz
Q15	Scope demonstration

Also this week: **clone your GitHub repo** to your local machine.

Tips for the Assignment

1. **Read the comment above each question** before writing any code
2. Use `console.log()` often — check output as you go, not just at the end
3. **For Q8, Q11, Q12:** use `.slice()` to copy arrays before modifying them
4. **For Q14 (FizzBuzz):** check divisible-by-both first — order matters!
5. **Stuck?** Ask an AI for hints, not answers:

"Can you give me 3 hints for this problem without showing me the solution?"

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel with loops and arrays?

Before next session:

- Read through the **lesson materials** (loops, arrays, scope, Git)
- Complete the **15-question assignment**
- **Clone** your GitHub repo to your local machine (lesson video walks you through it)
- Post questions in **Slack** anytime — we are here to help!

Resources

Resource	What it's for
MDN — Looping Code	Loops reference + active learning
JavaScript.info — While and For	Loop practice with tasks
W3Schools — Arrays	Array basics
W3Schools — Array Methods	push, pop, slice, and more
W3Schools — Scope	Scope explained simply
Slack <code>#discussion</code> channel	Questions between sessions

Next week: JavaScript Objects!

Great work today!

Loops and arrays unlock a whole new level of programming.

Questions? Reach out on Slack anytime.